

课题01-BPE分词器

Contents

1	课题01 开题简报 · BPE 分词器	1
1.1	一、研究背景（这个问题历史上卡过谁）	2
1.2	二、研究问题（一句话，开放问题）	2
1.3	三、攻关线索（路标，不是答案）	2
1.4	四、里程碑（可勾选，3-6 个）	2
1.5	五、交付物要求	3
1.6	五之二、官方判分标准（2026-09-17 实跑探明，先知道标准再动手）	3
1.7	六、开题批（开场直接发给用户，3-5 题）	4
1.8	七、讲课锚点（知行合一的“讲”这一腿）	5
1.9	八、参考实现（只准用于对照，不准抄）	5
2	判卷记录 · 课题01 BPE 分词器	6
2.1	L1 · 基线（2026-09-17 立科时实跑）	6
2.2	L2 · 导师审稿意见	6
2.3	L3 · 费曼收口	7
2.4	本课题的判分环境坑（已解决，记录备查）	7

1 课题01 开题简报 · BPE 分词器

模块：M2 分词与表示 | 周次：W1（09-14→09-20） | 算力：
T0 纯 CPU，0 GPU·小时 唐杰作业对应：① 从零写 `Tokenizer` 的前半 | 判分来源：CS336 A1 `tests/test_tokenizer.py` + `test_train_bpe.py` 手写/代写：本课题全部核心代码属手写区 R1（见《造轮子边界.md》）

1.1 一、研究背景（这个问题历史上卡过谁）

- 2017 年前的 NLP 用整词表，未登录词（OOV）是死穴；字符级建模又让序列变得极长。
- BPE 从 2016（Sennrich 等）的机器翻译子词切分技术，被 GPT-2（2019）引入到字节级，从此成为大模型的事实标准（GPT-2/3/4、LLaMA、Qwen、GLM 都是 BPE 系）。
- 关键洞察：词表不是“词汇表”，而是一个可学习的压缩算法——它同时决定了 ①模型的输入长度 ②嵌入层参数量 ③模型能见到的最小语义单元 ④多语言/代码/数学的公平性。

1.2 二、研究问题（一句话，开放问题）

在固定词表预算下，如何设计一个训练-编码两阶段都正确、且对中文/英文/代码都公平的子词分词器？——并回答：为什么“字节起步”是零预算下唯一不会翻车的选择？

1.3 三、攻关线索（路标，不是答案）

1. 压缩视角：如果词表大小为 V ，BPE 训练在最大化什么？把它写成“每一步减少多少 token 数”。
2. 字节起步的理由：从 Unicode 字符起步会在哪些具体输入上崩？（想 3 个真实例子再往下走）
3. 训练算法：朴素实现为什么是 $O(n^2)$ ？哪些数据结构能把它压到近似线性？（提示方向：不要用字符串拼接）
4. 并列与终止：最高频并列时怎么办？什么时候停止合并？特殊 token（`<|endoftext|>`）为什么不能参与合并？
5. 编码与解码：encode 的时候，遇到未在 merge 表里的相邻对怎么处理？为什么必须保证 $\text{decode}(\text{encode}(x)) = x$ （往返一致性）？
6. 多语言公平性：同一段中文，字节级 BPE 的 token 数为什么比英文多？这对模型的“中文成本”意味着什么？

1.4 四、里程碑（可勾选，3–6 个）

- ☐ **M1** 手写 BPE 训练：能从语料统计词频、迭代合并、输出 merge 表与词表，并打印出前 10 个 merge 及其频次
- ☐ **M2** 手写编码器：支持特殊 token、未登录字节回退、往返一致性（自测 5 类边界：空串/单字节/中文/emoji/控制字符）
- ☐ **M3** 通过官方判分：test_tokenizer.py 与 test_train_bpe.py 全绿（原始输出贴 判卷记录.md）
- ☐ **M4** 压缩率对照：在 TinyStories（英）与中文小语料上各训一个 32k 词表，出压缩率表 + 与你的下注对照

- **M5** 改坏实验（必做）：①禁用 bytes 回退 ②不处理特殊 token ③停止条件改成“合并到只剩 1 个词”——每个先写预测，再跑，再记录现象
- **M6** 一页报告 + 知识卡：报告按一页四段；知识卡 ≥ 3 张进 drill-ledger

1.5 五、交付物要求

交付物	位置	验收
实现代码	手写/（你写）	通过 M3
判分原始输出	判卷记录.md	粘贴 证据/judge-*.log 原文（不许转述）
证据	证据/	run.yaml（commit/时间/命令）+ 日志 + 压缩率表 + 改坏实验记录
报告	报告.md	一页四段：问题-动机-方法-结果
知识卡	memory/drill-ledger/ topics/	≥ 3 张（bytes 回退 / merge 规则 / 压缩率权衡）

1.6 五之二、官方判分标准（2026-09-17 实跑探明，先知道标准再动手）

判分器已接通：环境/judge/judge.sh 01。实跑探明 28 个测试（27 项 + 1 项 xfail），要求如下：

1.6.1 接口契约（你唯一需要遵守的外部形状）

```
# 手写/bpe.py 必须提供：
def train_bpe(input_path, vocab_size, special_tokens, **kwargs) -> tuple[dict
    [int, bytes], list[tuple[bytes, bytes]]]
class Tokenizer:
    def __init__(self, vocab, merges, special_tokens=None): ...
    def encode(self, text: str) -> list[int]: ...
    def decode(self, ids: list[int]) -> str: ...
    def encode_iterable(self, iterable) -> Iterator[int]: ... # ← 隐藏要求，见下
```

1.6.2 三条硬门槛（都会挂人）

#	测试	门槛	说明
1	test_train_bpe_speed	corpus.en + vocab 500 < 1.5 秒	官方参考 0.38s；“玩具实现”约 3s 会挂
2	test_train_bpe	merges 逐项等于参考 merges；vocab 键值集合一致	意味着预处理与并列打破规则都要与参考一致
3	test_encode_iterable	memory_usage encode_iterable 流式读 5MB 语料，额外内存 ≤ 1MB（RLIMIT_AS 实测）	必须有真正的流式实现，不能先 read() 全部

1.6.3 一个官方“故意让分”的测试（别被骗）

- test_encode_memory_usage 被官方标记 xfail，理由原话：“*Tokenizer.encode is expected to take more memory than allotted (1MB)*”。
→ 也就是说：encode() 超内存是允许的，encode_iterable() 超内存是不允许的。这个 xfail 恰好是本课题“接口设计决定内存上界”的活教材（见知识地图 M5 的同类问题）。

1.6.4 对齐口径的隐含要求

- 与 tiktoken 的 GPT-2 逐 id 对齐 → 编码时必须严格按 merges 的 rank 顺序合并，而不是“最长匹配优先”；且需处理 allowed_special 与重叠特殊 token（test_overlapping_special_tokens）。
- 往返一致性覆盖：空串 / 单字符 / Unicode / 多行 / 特殊 token 尾随换行 / 地址文本 / 德语 / TinyStories。

1.7 六、开题批（开场直接发给用户，3–5 题）

见 ../memory/HANDOFF.md 的“课题01 开题批”五题（下注题 / 概念题 / 手算题 / 权衡题 / 工程题）。第 1 题（中文 token 倍数）必须先存档预测，它是本学科第一条“预测→打脸”链路。

1.8 七、讲课锚点（知行合一的”讲”这一腿）

- 讲什么：M2 的五个核心问题（压缩视角 → 字节起步 → 训练算法 → 编码与往返 → 多语言成本）
- 对标材料（三师制）：
 - 主线：CS336 A1 handout 的 tokenizer 章节（英文，规格权威）
 - 中文：李沐 d2l ch14.6 子词嵌入；或 minimind 的 train_tokenizer.py 中文注释（只读，不得抄进手写区）
 - 补充：HF LLM Course ch2（“Implement BPE/WordPiece/Unigram from the ground up”）
- 讲完必出代码级提问（3–5 道，示例）：
 1. 把 merge 循环里”统计相邻对频次”改成”只统计出现次数 >1 的对”，词表会变得更好还是更差？预测并说明。
 2. 你的 encode 如果按”最长匹配优先”而不是”按 merge 顺序”，往返一致性还成立吗？
 3. 词表里如果混入一个重复的 token，哪一步会先崩溃？

1.9 八、参考实现（只准用于对照，不准抄）

- karpathy/nanochat 的 nanochat/tokenizer.py（接口设计与特殊 token 处理）
- rasbt/LLMs—from-scratch ch2 附带的 “BPE From Scratch”
- △ 纪律：GS=看接口与设计思路，不看实现细节；你的代码必须自己写。判据：M5 的改坏实验能不能被你解释——抄来的人在这里一定卡住。

2 判卷记录 · 课题01 BPE 分词器

判分器：环境/judge/judge.sh 01（CS336 官方 2026 测试，锁 commit a158843b2010, test_*.py 未改一字）归档规则：只贴原始输出，不做美化、不做转述（《判分与验收标准.md》§二-3）。

2.1 L1 · 基线（2026-09-17 立科时实跑）

项	结果
命令	环境/judge/judge.sh 01
结果	27 failed, 1 xfailed
失败原因	全部同一原因：FileNotFoundError： 手写区还没有实现文件（接线层主动抛出的友好提示）
判分器状态	✅ 管线已验证可执行（28 个测试全部被收集并运行，非网络/环境错误）
结论	这是正确的初始状态：判分器就位，只差 手写/bpe.py

当初实跑的关键片段（完整输出会由脚本落到 证据/judge-*.log）：

```
FAILED tests/test_train_bpe.py::test_train_bpe_speed - FileNotFoundError
FAILED tests/test_train_bpe.py::test_train_bpe - FileNotFoundError
FAILED tests/test_train_bpe.py::test_train_bpe_special_tokens -
FileNotFoundError
FAILED tests/test_tokenizer.py::... (24 项, 含
test_encode_iterable_memory_usage)
===== 27 failed, 1 xfailed in 3.67s
=====
```

注：1 xfailed 不是缺陷——它是官方故意标记的 test_encode_memory_usage（理由：“encode 预期会超过 1MB 配额”）。详见 开题简报.md §五之二。

2.2 L2 · 导师审稿意见

🕒 待用户提交首次实现后填写（3-5 条：必考项 / 要记住 / 预告项）。

2.3 L3 · 费曼收口

□ 未开始。收口动作：你对着”完全不懂分词的人”讲清 BPE，我装不懂连问三层（Why → Why not → What if），再抽 1 次”改坏实验”复述。

2.4 本课题的判分环境坑（已解决，记录备查）

坑	现象	处置
官方 conftest 需 Python ≥ 3.12	官方 pyproject.toml: <code>requires-python = ">=3.12,<3.14"</code> ；conftest 用 PEP 695 泛型语法，3.11 无法解析	3.11 下自动改用等价轻量 conftest （同一 snapshot 夹具语义），并在日志头部如实声明；升到 3.12+ 后自动切回官方原文
tiktoken 需联网下载 GPT-2 词表	无外网时 <code>SSLError</code> 挂掉 12 项对齐测试	新增环境 <code>/judge/prefetch/_tiktoken.py</code> ：用官方 fixture 离线构造 tiktoken 缓存（ <code>encoder.json</code> 哈希与 tiktoken 内置期望一致，已验证 <code>n_vocab=50257</code> ）
PyTorch CPU 专用源不可达	<code>download.pytorch.org</code> SSL 被拦	<code>bootstrap</code> 改为：默认不装 torch（课题01 不需要）； <code>--with-torch</code> 时先试 CPU 源、失败退回 PyPI